

## C programming Function Assignment

### 1. Problem Statement

Create a C program that acts as a menu-driven calculator. The program should offer a user a list of options to perform various mathematical and geometric calculations.

The program should meet the following requirements:

1. **Menu Display:** Present a clear, numbered menu of options to the user.
2. **User Input:** Prompt the user to select an option from the menu.
3. **Calculation Functions:** Use separate functions for each calculation to ensure modularity. The functions should handle the following operations:
  - Calculate the area of a **circle** given its radius.
  - Calculate the area of a **rectangle** given its length and width.
  - Calculate the area of a **square** given its side length.
  - Calculate the **power** (xy) of a number.
  - Calculate the **square root** of a number.
4. **Looping:** The program should continuously display the menu and perform calculations until the user explicitly chooses to exit.
5. **Exit Condition:** An option must be available for the user to gracefully terminate the program.
6. **Error Handling:** The program should handle invalid user input (e.g., a choice not on the menu) and display an appropriate error message. It should also handle specific calculation errors, such as attempting to find the square root of a negative number.

The solution must use **functions** for each calculation, utilize a looping structure (such as while or do-while) for the menu, and employ a **switch-case** statement to execute the chosen operation.

### 2. Problem Statement

Create a C program for a number guessing game. The program will randomly select an integer between 1 and 100, and the user must guess it. The game should provide hints after each guess until the user finds the correct number.

The program must be structured with the following key components:

1. **Random Number Generation:** Use the <stdlib.h> and <time.h> libraries. The random number generator must be seeded with `srand(time(0))` to ensure a different number

is chosen each time the program runs. The target number should be an integer between 1 and 100, generated using  $(\text{rand}() \% 100) + 1$ .

2. **User Input Function:** Implement a function called `getUserGuess()` that is solely responsible for prompting the user for an integer guess and returning it.
3. **Guess Evaluation Function:** Implement a function called `checkGuess()` that accepts the user's guess and the target number as arguments. This function should compare the two numbers and print a hint ("Too high!" or "Too low!"). It should return a flag (e.g., 1 for a correct guess, 0 for an incorrect one) to control the game loop.
4. **Game Loop:** The `main()` function should contain a loop that repeatedly calls the `getUserGuess()` and `checkGuess()` functions until the correct number is guessed.
5. **User Exit Option:** The program should allow the user to exit the game at any time by entering a specific number, such as 0. When the user enters this value, the program should terminate gracefully without proceeding with the guess check.

The program should terminate gracefully once the user has either correctly guessed the number or chosen to exit the game.